

Table of Contents

1. Executive Summary	1
2. Objective, Scope and Phase Ledger	2
3. Customer Workflow	3
4. Provider Workflow	3
5. Job Request State Machine	4
6. Request Form, Location and Attachments	5
7. Provider Response, Notifications and Messaging	6
8. API and Database	7
9. Security and Audit Trail	8
10. Test Results	9
11. Acceptance Tests	10
12. Browser End-to-End, Performance and Known Issues	11
13. Deferred Features, Git and the Phase 6 Recommendation	12

1. Executive Summary

94 Phase 5 verification checks, all passing	377 Total checks across the five suites	26 / 26 Browser E2E steps across three viewports	10 / 10 Critical acceptance tests passing
--	--	---	--

Phase 5 of the Dwellers platform completed the transformation that the product story has demanded since the beginning: the Request Service button now creates a real job request in the database, the targeted provider genuinely receives and responds to it, and the customer sees that response in a timeline built from real events. Building strictly on the Phase 1 security foundation, the Phase 2 domain backend, the Phase 3 authentication layer and the Phase 4 discovery engine, the phase delivered the full journey the owner's brief specified: FIND, SELECT PROVIDER, DESCRIBE JOB, ADD LOCATION, ADD PHOTOS, SUBMIT, PROVIDER RECEIVES REQUEST, PROVIDER RESPONDS, CUSTOMER SEES RESPONSE. The work was performed against the 86-part Phase 5 requirements brief, and every part of that brief is accounted for in this report. Nothing from Phases 1 to 4 was rebuilt, replaced or duplicated - no second authentication system, no second messaging, location, provider or quotation system.

Three design decisions define the phase. First, the state machine is central: a client never submits a status, only an action, and a single server module decides the resulting state - so a customer forcing status = ACCEPTED through a PATCH request is stripped at the validation boundary and refused at the machine. Second, everything is atomic and honest: every transition writes its timeline event, its audit record and its notification inside one database transaction, and a failed submission never tells the customer that it succeeded. Third, privacy is structural: the provider sees a customer display name only, job photos stream through an authorization-checked route instead of public storage URLs, and job requests never appear in public APIs, the sitemap or SEO pages.

The proof is behavioural. The new verify-phase5 suite adds 94 checks on top of the Phase 1 (48), Phase 2 (106), Phase 3 (73) and Phase 4 (56) suites for a combined battery of 377 checks, all green, including the ten critical acceptance tests of the brief: the Raymond-to-Kwame Nsawam plumbing request with three photos; the provider receiving, viewing and responding; the customer seeing the response; rejection of a service the provider does not offer; rejection of an invalid region/district/town combination; double-click protection; the IDOR matrix for customers, providers and attachments; the client-controlled-status refusal; and the suspended-provider guard. A clean-database rebuild was executed again this phase, and a 26-step browser E2E walked the complete customer-to-provider-to-customer journey at 390, 768 and 1440 pixel widths with zero console errors and zero horizontal overflow. The project now sits at the Phase 6 gate, Quotations, awaiting the project owner's explicit approval.

2. Objective, Scope and Phase Ledger

2.1 Objective

The Phase 5 brief states the objective directly: transform the existing Request Service button into a REAL job-request workflow. Before this phase the button honestly announced that requests arrive with the next release; nothing was faked, and nothing was stored. After this phase, the button opens a six-step request wizard that creates a durable JobRequest row, notifies the provider, records every lifecycle event, and gives both sides a professional tracking surface. Thirteen numbered customer capabilities from the brief - find, select, describe, locate, detail, attach, schedule, review, submit, confirm, track, provider receipt, provider response, customer visibility - are each implemented and each covered by an automated check.

2.2 Scope discipline

The phase reused, extended or at most touched four seams of the existing architecture, and built nothing parallel. The Prisma schema gained one model and five columns, all documented below; no existing model changed meaning. The job-request service module kept its role as the single owner of request logic. The API route tree kept its established handler factory, envelope and error conventions. The dashboard shells, navigation configuration and discovery components were extended, not replaced. The quotation system, messaging conversations, payments and analytics dashboards of later phases were left exactly where Phase 4 put them: as ready foundations with clean seams.

2.3 Phase ledger

Phase	Scope	Status
1	Foundation, architecture and standards	Complete (gate: PASS WITH CONDITIONS)
2	Database schema and backend foundation	Complete
3	Authentication, registration, role dashboards	Complete
4	Find / Nationwide Discovery	Complete
5	Request Service / Job Requests	Complete - this report
6+	Quotations, payments, messaging, commerce	Awaiting owner approval

Per the project's gate discipline, Phase 6 has not been started. The final section of this report records the recommendation for that phase and nothing more.

3. Customer Workflow

The customer journey begins exactly where Phase 4 ended. A customer searching Plumbing in Nsawam reaches the provider results page, opens Kwame's profile, and presses Request service. If the customer is anonymous, the click routes to sign-in with the destination and the intended action encoded in the callback URL - the browser E2E walked this round trip and verified that after authentication the customer lands directly inside the request wizard for the chosen provider, with the intent never lost. If the customer is already signed in, the click goes straight to the wizard.

The wizard at `/request-service/[providerId]` is a focused multi-step form, never one endless page. Step one lists the services the provider genuinely offers, with the honest constraint that no other service can be chosen; the server re-validates this on submission, so even a forged request naming a service the provider does not list is rejected. Step two collects the job title and description with live character counters and the brief's limits of 10 to 120 and 10 to 5,000 characters. Step three selects the job location through the existing Region, District, Town cascade plus an optional community list and a free-text landmark field. Step four captures the schedule: as soon as possible, a specific date that cannot be in the past, or flexible, together with a preferred time slot and an urgency of normal, urgent or emergency. Step five uploads photos. Step six renders the complete review, and only the server's confirmation moves the customer to the confirmation page.

A draft exists from the moment the wizard opens. This is deliberate: photos must upload against a real request row through the established storage pipeline before submission, and a draft is also the honest answer to abandonment - the customer sees the draft in My Job Requests and can continue or cancel it. Draft creation is idempotent on a browser-generated client token held in session storage, so a page refresh or a retried request resolves to the same draft instead of piling up duplicates. The submission itself carries all fields plus the submit action; the server re-validates everything and only a confirmed response navigates the customer to the confirmation view with the human-friendly reference, the status and the next step.

4. Provider Workflow

Providers receive requests through the dashboard section Job Requests, now live for every provider role: artisan, contractor, construction company, supplier and equipment provider. The board organises work the way the brief describes - New, Active, Completed, Declined and Cancelled sections, expanded server-side from the state machine so the client cannot invent a section or widen a scope. Each row shows the request reference, the service, the job title, the job location, the requested timing, the photo count and the status, with urgent and emergency requests visually flagged.

Opening a request shows everything a provider needs to decide: the service, the full description, the location with the landmark text, the preferred timing, the urgency, the photos, and the customer as a display name only - first name and last initial, never an email or phone number. The first provider-side open records the real VIEWED event exactly once, so the customer's timeline later shows when the provider actually looked. Where the job location falls outside the provider's listed service areas, the page says so plainly rather than pretending otherwise.

The response panel offers the brief's three actions - I am interested, Ask for more info, and Decline - with an optional message of up to one thousand characters. Responding is a single server action: the status moves through the state machine, the response kind and timestamp are stored, the message becomes a timeline entry, the customer is notified inside the same transaction, and the panel is replaced by an honest record of the response that was made. A second response attempt is refused by the machine. Business-affiliated providers see the same surface, but the authorization layer grants the respond actions only to active owner or manager members; a plain member receives a read-only explanation instead of buttons that would fail.

5. Job Request State Machine

The state machine lives in one module, `src/modules/projects/job-request-state.ts`. It preserves the Phase 2 status vocabulary - DRAFT, SUBMITTED, MATCHING, RESPONDED, ACCEPTED, IN_PROGRESS, COMPLETED, CANCELLED - and adds one status the brief's provider dashboard requires: DECLINED, the real, terminal outcome of a provider saying no. Every action a client can submit is an entry in a central table; every state change in the service layer passes through the same `assertTransition` guard. There is no code path anywhere in the application that sets a status from client input.

Action	Actor	Permitted from	Resulting status
submit	Customer	DRAFT	SUBMITTED
edit	Customer	DRAFT	unchanged
cancel	Customer	DRAFT, SUBMITTED, MATCHING, RESPONDED	CANCELLED
respond_interested	Provider	SUBMITTED, MATCHING	RESPONDED (INTERESTED)
respond_info	Provider	SUBMITTED, MATCHING	RESPONDED (NEEDS_INFO)
respond_declined	Provider	SUBMITTED, MATCHING	DECLINED

Permitted transitions (the complete set)

The machine's security properties were tested directly. A customer's respond actions are structurally impossible because no customer source states exist for them; a provider cannot act on a completed, cancelled or already-responded request; staff cannot perform any business transition at all; and a cancelled request can never be re-accepted. At the HTTP boundary the same properties were confirmed end to end: a PATCH carrying status = ACCEPTED has the unknown key stripped by validation, the remaining edit refused because the request is not a draft, and the stored status verified unchanged by a fresh read.

Terminal states are terminal: COMPLETED, DECLINED and CANCELLED allow no further transitions of any kind. The customer's cancel path deliberately remains open while a request waits for a provider or carries an early response, and closes once the engagement has committed further - the exact cut-off the brief asks for, enforced server-side with a confirmation step in the interface.

6. Request Form, Location and Attachments

6.1 The form and its fields

The wizard's complete field set, with the validation each field carries on both sides of the wire, is listed below. Nothing asks the customer for technical knowledge; nothing asks for contact details the account already holds; and every limit in the brief is enforced by the server regardless of what the browser allows.

Field	Required at submit	Rules
Service	Yes when a provider is targeted	must be offered by that provider, active and available
Title	Yes	10-120 characters, trimmed
Description	Yes	10-5,000 characters, control characters rejected
Job location (town)	Yes	must exist and be active in the Ghana hierarchy
Community / area	Optional	must belong to the chosen town - hierarchy validated server-side
Address / landmark	Optional	up to 200 characters of free text
Preferred timing	Optional	ASAP, or a date not in the past, or flexible; time slot; urgency
Budget range	Optional	integer pesewas; maximum cannot be below minimum
Photos	Optional	up to 10 JPEG/PNG/WEBP files, 10 MB each

Request fields

6.2 Job location and the Ghana hierarchy

The job location is chosen through the live location API - Region, District, Town, then an optional community list - and it is explicitly independent of the customer's own profile location, exactly as the brief requires: a customer living in Accra can need a plumber in Nsawam. Cascading selectors are never trusted: the server re-resolves every reference and rejects a community that belongs to a different town, a town that does not exist, or a community submitted without a town. When the chosen town lies outside the provider's listed service areas the interface shows an honest notice and the request is still permitted, because the provider - not the system - decides whether to take the job.

6.3 Photos and their security

Uploads use the existing storage abstraction with a dedicated job-attachment policy: JPEG, PNG and WEBP only, ten megabytes each, SVG categorically blocked as a stored-XSS vector, filenames sanitised and keys generated as category-scoped UUIDs so client data never influences the physical path. The cap of ten photos per request is enforced against the live attachment count. The photo bytes are served back only through an authorization-checked route that verifies the caller is the owning customer, the targeted provider side or staff - storage keys are never exposed in any payload, and foreign probes of an attachment receive the same opaque 404 as any other missing resource. Removing a photo deletes the row and the binary and writes the real ATTACHMENT_REMOVED event.

7. Provider Response, Notifications and Messaging

7.1 The response actions

The response workflow is the first provider-side decision surface in the product. Interested, ask-for-more-info and decline each map through the central machine to a distinct outcome, and each accepts an optional message that becomes part of the permanent timeline. The response is stored with the acting user and role; the customer's view renders the response card with the exact message text; and the request's responseKind column gives dashboards their honest grouping. Because responding implies reading, the first response also fixes the viewed timestamp if the provider somehow never opened the request.

7.2 Notifications

Two in-app notification types were added to the existing notification architecture - no new table, no new system. When a request is submitted, the targeted provider user receives New service request in Nsawam, with the service named and the request linked. When the provider responds, the customer receives one of three wordings: accepted, needs more information, or declined. Both notifications are created inside the same database transaction as the change that caused them, so a state change can never exist without its

notification. Email, SMS and WhatsApp remain deliberately absent - the Notification model's channel column and sentAt bookkeeping are already schema-ready for those later integrations, and no external provider is a dependency of this phase.

7.3 Messaging foundation

The phase uses the existing communication foundation where it genuinely helps and leaves full conversations where the roadmap already placed them. The provider's ask-for-info response carries its message as a timeline entry visible to the customer, which answers the brief's request-more-information flow without inventing a second messaging model. The profile's Message provider button remains an honest placeholder that sends nothing, and the dashboard navigation continues to mark full messaging for Phase 6. Conversation and ConversationParticipant models - with their jobRequestId link - are untouched and ready.

8. API and Database

8.1 Endpoints

Eight endpoints carry the phase, built on the established handler factory with its fixed pipeline: rate limit, authentication with a live account-status re-check, RBAC, Zod validation, ownership, business rules, database transaction, audit event and the standard response envelope. The two Phase 2 routes were extended rather than replaced, and the brief's suggested submit and cancel paths remain actions on the existing PATCH route exactly as the brief permits.

Endpoint	Change	Purpose
POST /api/job-requests	extended	idempotent create; draft or direct submit; returns authorized detail
GET /api/job-requests	extended	role-scoped list with status and statusGroup filters
GET /api/job-requests/[id]	extended	detail + timeline; provider side marks VIEWED once
PATCH /api/job-requests/[id]	extended	customer actions: edit, submit, cancel
POST /api/job-requests/[id]/respond	new	provider response actions with optional message
POST /api/job-requests/[id]/attachments	new	multipart photo upload through the storage policy
GET /api/job-requests/[id]/attachments/[attachmentId]	new	authorization-checked photo streaming
DELETE /api/job-requests/[id]/attachments/[attachmentId]	new	customer removes a photo while editable

Phase 5 API surface

8.2 Schema changes

The migration `phase5_job_request_lifecycle` is deliberately small. The `JobRequest` table gains five nullable columns: `urgency`, `viewedAt`, `respondedAt`, `responseKind` and `clientToken`, the last with a unique index that makes duplicate submission impossible at the database level. A new `JobRequestEvent` table stores the append-only timeline with an index on request and creation time. The attachment table gains an `uploadedById` audit column and defaults its kind to `PHOTO`. No existing column changed type or meaning, no model was duplicated, and the clean-database rebuild test applied all five migrations from zero and reseeded without error.

8.3 Money

The existing money policy stands unchanged: the Ghanaian currency is recorded as GHS, amounts are integer pesewas, floating point is never used, and the optional budget range on a request is validated so the maximum cannot sit below the minimum. No new money abstraction was introduced, and quotation pricing remains entirely outside this phase.

9. Security and Audit Trail

9.1 The security review, point by point

The brief's security review lists fifteen areas; each is enforced by structure and each was exercised by a test. Authentication guards every endpoint, with the live database re-check suspending an account mid-session. RBAC keeps projects:create customer-only, so providers cannot file requests at themselves. Ownership is resolved through one service-level resolver that returns the caller's side - customer, provider or staff - and the `canManage` decision for business members; everyone else receives the same 404 an anonymous visitor would get. State transitions belong to the central machine. Validation strips unknown fields and rejects malformed identifiers with 422s. Rate limiting uses the standard preset on request routes, the upload preset on attachments and the auth preset on sign-in, returning 429 with `Retry-After`.

The PART 65 IDOR matrix was executed over live HTTP with real sessions for every row: customer A viewing, editing, cancelling or attaching to customer B's request; customer A touching the provider response endpoint; provider A viewing or responding to provider B's request; a provider deleting a foreign attachment; and a business member attempting protected actions. Every attempt returned 404 or 403, never 200 and never 500, and no response was created. A request enumeration probe through the public discovery API and the sitemap found no request identifiers and no private URLs.

9.2 The audit trail

The append-only audit log gained nine job-request actions: `created`, `submitted`, `updated`, `viewed`, `responded`, `declined`, `attachment_added`, `attachment_removed` and `cancelled`. Each records the actor, role,

entity and a minimal metadata object; passwords, secrets and unnecessary personal data are excluded by the existing redaction pipeline, and audit failures can never break the business operation. The verification suite asserts that every one of the nine actions appears for the fixture lifecycle.

10. Test Results

94 verify:phase5 checks	377 checks across all five suites	0 failures anywhere
-----------------------------------	---	-------------------------------

The verification architecture follows the established two-level pattern. The service level runs against an isolated database rebuilt from the migration files, exercising the state machine, validation rules, transactions, notifications, audit, analytics, attachments, business permissions and suspension guards directly. The HTTP level runs against the live development server with real NextAuth sessions, real middleware and the seeded demo database, exercising the acceptance journeys, the IDOR matrix, upload limits, rate limiting and privacy over the wire. Nothing is mocked: every check goes through the same code paths the product uses.

Suite	Checks	Result
verify (foundation)	48 / 48	pass
verify:phase2	106 / 106	pass
verify:phase3	73 / 73	pass
verify:phase4	56 / 56	pass
verify:phase5	94 / 94	pass
Total	377 / 377	pass

Combined verification battery (this phase's fresh run)

Two suites required small, honest updates this phase and both are recorded here. The Phase 2 suite's draft fixture now carries a title and location before submission, because Phase 5 enforces the brief's submit-time completeness rules that Phase 2 predated. The Phase 5 suite itself gained three defect-driven checks during development, described in the Known Issues chapter: the envelope shape of the list endpoint, the oversized-upload status code and the binary streaming route. TypeScript passes with zero errors and ESLint with zero warnings; both are part of the completion standard and were run fresh before this report.

11. Acceptance Tests

The brief defines ten critical acceptance tests. All ten were automated - nine in `verify:phase5` at both service and HTTP level, and the full browser journey as the 26-step end-to-end walk reported in the next chapter. The table records each test with its expected outcome and its actual result in this phase's fresh runs, including on the clean-database rebuild.

#	Test	Expected	Result
1	Raymond files Fix leaking bathroom pipe, Plumbing, Nsawam, photos, ASAP	one real SUBMITTED request with all data persisting	pass
2	Provider opens Job Requests, views everything, responds with a message	response stored, status per machine, customer notified	pass
3	Customer opens My Job Requests and the request	response visible, timeline updated	pass
4	Electrical submitted to the plumber	server rejects; no request created	pass
5	Invalid region/district/community combination	422 with field error; nothing created	pass
6	Double-click submit	exactly one request exists	pass
7	Customer A reads customer B's request	404, never 200	pass
8	Provider A responds against provider B's request	404; no response created	pass
9	Customer PATCH with status = ACCEPTED	rejected; server owns state	pass
10	Suspended provider attempts to act	current account rules applied; no protected action	pass

The ten critical acceptance tests

Acceptance test one deserves its sentence: on a freshly migrated and seeded database, Raymond filed the leaking-bathroom-pipe request against Kwame's plumbing service in Nsawam with photos and an as-soon-as-possible schedule; one `JobRequest` row was created with status `SUBMITTED` and reference `JR-Y3FCNUXU` on the clean-database run; the provider, service, location, urgency and attachments all persisted across a server restart and a page refresh.

12. Browser End-to-End, Performance and Known Issues

12.1 The 26-step browser journey

The brief's end-to-end script was walked in a real browser against the live application: open Dwellers, search Plumbing, select Nsawam through the cascade, open the provider, press Request service, sign in as the customer, land in the wizard, fill the title and description, select the location, choose the preferred timing, upload a photo, review, submit, see the confirmation with the reference, open My Job Requests, open the request details; then as the provider - sign in, open Job Requests, open the customer's request, view the job details and photos, respond with a message; then as the customer again - return to the request, see the provider's response and the updated status. All twenty-six steps passed, and screenshots of the run are committed alongside the test evidence.

12.2 Responsive and console checks

The customer pages, the wizard, the results page and the home page were swept at 390, 768 and 1440 pixel widths measuring horizontal overflow directly from the DOM. The sweep found one genuine defect: the site header showed its desktop navigation at 768 pixels, pushing the page three hundred and nine pixels wide. The fix moved the desktop navigation to the large breakpoint, where the mobile anchor strip takes over below it, and the same sweep then reported zero overflow at every viewport on every page. The browser console was captured across all key pages at the end of the run: zero errors.

12.3 Performance

The phase adds one indexed table and five nullable columns; every hot query is served by existing or new indexes - request lookups by primary key, provider boards by the new provider-and-status index, timelines by the request-and-time index, and the unique clientToken index turns duplicate protection into a single lookup. List endpoints paginate through the shared clamped pagination contract and select only the columns the interface renders; the detail route loads the request, its events and its attachments in a fixed number of queries with no N-plus-one pattern. The heaviest transaction - submission with attachments, events, and provider notification - completed in tens of milliseconds on the development hardware, and the response transaction behaves the same.

12.4 Known issues

Issue	Found by	State
Provider board read the list envelope as an object; the data is the array - crashed after first fetch	browser E2E	fixed + regression-tested
Oversized upload bodies exceeded the framework cap and surfaced as 500	verify:phase5	fixed: honest 413
Photo streaming route returned a bare Response, which the factory wrapped as JSON	verify:phase5	fixed: NextResponse passthrough
Customer display initials mishandled parenthesised name suffixes	browser E2E	fixed
Header navigation overflowed 768-pixel viewports	responsive sweep	fixed
Avatar upload route shares the oversized-body failure mode of 8.2	code review	recorded, pre-existing, untouched by this phase

Real issues found during verification, and their state

13. Deferred Features, Git and the Phase 6 Recommendation

13.1 Deliberately deferred

The brief's do-not-build list is respected in full: no payment gateway, checkout, escrow or quotation marketplace; no WhatsApp, SMS or advanced email automation; no equipment rental transactions, project management, AI recommendations or construction cost estimator; no advanced analytics dashboard. The Quote model remains exactly as Phase 2 defined it, ready for the next phase. Within messaging, the request-scoped response message is the extent of the surface, and full conversations stay deferred. Saved providers, favourites and review flows also remain on their roadmap rows.

13.2 Git state

The phase was delivered in four reviewed commits plus one evidence commit: the engine (state machine, lifecycle service, respond and attachment APIs, migration), the frontend (wizard, customer and provider surfaces, navigation, header fix), the tests (the 94-check suite, seed fixtures, the adapted Phase 2 fixture) and the documentation (JOB_REQUESTS.md plus the architecture, database and readme updates), followed by the end-to-end screenshots. The working tree is clean of unstaged changes, the history is linear, and the branch is pushed to origin with head at e2dc074.

13.3 Phase 6 recommendation

The natural next phase is Quotations: JobRequest to Provider Response to Quote to Customer Reviews Quote to Accept or Decline. The foundation is genuinely ready - the Quote and QuoteItem models with

their server-computed totals in integer pesewas, the RESPONDED state whose responseKind distinguishes interested providers from the rest, the customer and provider surfaces that a quote thread attaches to, and the notification architecture that carries the next event. A quotation phase would let the provider price labour, materials, equipment and other costs against a specific request, keep totals server-computed under the existing money policy, and give the customer a side-by-side comparison surface. Payment integration should remain outside that phase: acceptance and escrow-shaped flows deserve their own gate.

13.4 Handover summary

What the owner receives with this phase is best expressed as the things that are now true. A customer with an account can find any provider in the country and ask them for service without a phone call; the request exists as a durable row with a shareable reference, a photo set, a verified location and a schedule. The provider sees that request in a dashboard built for decisions, responds once, and the response reaches the customer as a notification and a permanent timeline entry. Every step of that journey is guarded by authentication, role checks, ownership resolution and the central state machine; every step is auditable; and every step was proven twice over - once by 94 automated checks and once by a human-path browser walk.

Artifact	Location
State machine and lifecycle service	src/modules/projects/job-request-state.ts, job-request-service.ts
API surface	src/app/api/job-requests/** (8 endpoints)
Customer surfaces	/request-service/[providerId], /customer/requests, /customer/requests/[id]
Provider surfaces	/artisan /contractor /company /supplier /equipment - /requests and /requests/[id]
Verification suite	scripts/verify-phase5.ts via bun run verify:phase5 (94 checks)
End-to-end evidence	scripts/e2e-p5-*.png (10 screenshots of the 26-step run)
Documentation	JOB_REQUESTS.md; ARCHITECTURE.md; DATABASE.md section 8.2; README.md
This report	download/Dwellers_Phase5_Request_Service_Job_Requests_Report.pdf

Where the deliverables live

Per the gate discipline observed on every phase: Phase 6 has not been started. The project waits for the project owner's explicit approval and instruction.